Task 1: Concurrent Password Strength Checker Server

Create a multi-threaded TCP server that validates password strength for multiple clients simultaneously. The server uses artificial processing delays to demonstrate concurrent execution — when one client's password is being checked, other clients can still send requests and get processed in parallel.

Requirements

Server

1. Accept multiple client connections (minimum 3 clients).
2. Create a new thread for each client connection.
3. For each password received:
    - Apply a 5-second artificial delay (simulates complex hashing/validation).
    - Check password strength based on criteria.
    - Send result back to the client, including detailed breakdown (✓/✗ per criterion).
4. Each client session continues until that client sends "exit".
5. The server keeps running until manually terminated.
6. Print connection and processing status on the server terminal.

Client

1. Connect to the server via TCP socket.
2. Prompt the user to enter a password.
3. Send the password to the server.
4. Receive and display the strength result.
5. Continue until the user types "exit".

Password Strength Criteria

Criteria	Points
Length >= 8 characters	1
Contains uppercase letter	1
Contains lowercase letter	1
Contains digit	1
Contains special character (@, #, $, !, etc.)	1

Result Format

Score	Strength
0–2	Weak
3	Medium
4–5	Strong

Sample Output

Server Terminal

```
Password Strength Checker Server Started...
Listening on port 5000

Client1 connected from 127.0.0.1:54321
Client2 connected from 127.0.0.1:54322
Client3 connected from 127.0.0.1:54323

Client1: Checking password... (5 sec delay started)
```

```
Client2: Checking password... (5 sec delay started - parallel!)
Client3: Checking password... (5 sec delay started - parallel!)

Client1: Result sent - Strong (5/5)
Client2: Result sent - Weak (2/5)
Client3: Result sent - Medium (3/5)

Client1 disconnected.
```

Client1 Terminal

```
Connected to Password Checker Server

Enter password: MyP@ss123
Checking... (waits ~5 seconds)
Result: Strong (5/5)
Details: Length✓ Uppercase✓ Lowercase✓ Digit✓ Special✓

Enter password: weak
Checking... (waits ~5 seconds)
Result: Weak (1/5)
Details: Length✗ Uppercase✗ Lowercase✓ Digit✗ Special✗

Enter password: exit
Disconnected from server.
```

Client2 Terminal (running simultaneously)

```
Connected to Password Checker Server

Enter password: hello123
Checking... (waits ~5 seconds, but processes parallel to Client1!)
Result: Medium (3/5)
Details: Length✓ Uppercase✗ Lowercase✓ Digit✓ Special✗
```

Task 2: Concurrent Math Server using fork()

Create a concurrent TCP server that evaluates arithmetic expressions sent by multiple clients simultaneously. The server uses the fork() system call to spawn a new process for each client connection. This demonstrates **process-based concurrency** — while one client's request is being processed, other clients can still send requests and receive results in parallel.

Requirements

Server

1. Accept multiple client connections (minimum 3 clients).
2. For each new client:
    - Call fork() to create a child process.
    - The child handles all communication with that client.
    - The parent closes the client socket and continues accepting new clients.
3. For each expression received:
    - Parse the arithmetic operation (e.g., "12 + 7").

        - Perform the calculation.
        - Send the result back to the client.
    4. If the client sends "exit", terminate communication with that client.
    5. Print connection and processing status on the server terminal.

Client

1. Connect to the server via TCP socket.
2. Prompt the user to enter an arithmetic expression.
3. Send the expression to the server.
4. Receive and display the calculated result.
5. Continue until the user types "exit".

Expression Format

Input format:

number1 operator number2

Examples:

- 12 + 7
- 15 * 4
- 40 / 5

Supported operators:

- + (addition)
- - (subtraction)
- * (multiplication)
- / (division, division by zero must be handled as invalid)